

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104

Course Title: Data Structures Lab

Assignment No: 03

Date of Submission: 12/07/2026

Submitted by,

Name: Sheikh Zubayer Kabir

Student ID: 00724205101021

Lab Section: A1

Academic Semester: Fall 2025

Program: BSc in CSE

Question : Write a code for implementing Quick Sort and Merge Sort.

Solution:

Merge Sort:

The Merge Sort algorithm is a divide-and-conquer algorithm that sorts an array by first breaking it down into smaller arrays, and then building the array back together the correct way so that it is sorted.

Quick Sort :

Quicksort is a divide-and-conquer algorithm. It works by selecting a "pivot" element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. For this reason, it is sometimes called partition-exchange sort.

Task 1:

1. Given an array of N integers, simulate Quick Sort using the last element as pivot and

count the number of comparisons performed during the sorting process.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int compareCount = 0;
```

```
int Partition(vector<int> &arr, int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
    {
```

```
        compareCount++;
```

```

        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

```

```

void Quick_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = Partition(arr, low, high);
        Quick_Sort(arr, low, pivotIndex - 1);
        Quick_Sort(arr, pivotIndex + 1, high);
    }
}

```

```

int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;
}

```

```

vector<int> arr(N);
cout << "Please enter the elements of the array: " << endl;
for (int i = 0; i < N; i++)
    cin >> arr[i];

Quick_Sort(arr, 0, N - 1);

cout << "Number of comparisons: " << compareCount << endl;

cout << "Sorted array: " << endl;
for (int i = 0; i < N; i++)
    cout << arr[i] << " ";
cout << endl;

return 0;
}

```

/*Output

Please enter the size of the array:

6

Please enter the elements of the array:

5 2 8 1 9 3

Number of comparisons: 8

Sorted array:

1 2 3 5 8 9

*/

Task 2:

2. Given an array and a pivot, rearrange the array such that elements less than pivot come before it and greater come after.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int PartitionOnValue(vector<int> &arr, int low, int high, int  
pivotValue)
```

```
{
```

```
    int pivotPos = -1;
```

```
    for (int k = low; k <= high; k++)
```

```
    {
```

```
        if (arr[k] == pivotValue)
```

```
        {
```

```
            pivotPos = k;
```

```

        break;
    }
}
if (pivotPos == -1)
{
    cout << "Pivot value not found in array!" << endl;
    return -1;
}
swap(arr[pivotPos], arr[high]);

int pivot = arr[high];
int i = low - 1;

for (int j = low; j < high; j++)
{
    if (arr[j] < pivot)
    {
        i++;
        swap(arr[i], arr[j]);
    }
}
swap(arr[i + 1], arr[high]);

```

```
    return i + 1;
}

int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " << endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    int pivotValue;
    cout << "Please enter the pivot value: " << endl;
    cin >> pivotValue;

    int finalIndex = PartitionOnValue(arr, 0, N - 1, pivotValue);

    if (finalIndex != -1)
    {
```

```
        cout << "Array after partitioning around pivot " <<
pivotValue << ": " << endl;
        for (int i = 0; i < N; i++)
            cout << arr[i] << " ";
        cout << endl;
        cout << "Pivot placed at index: " << finalIndex << endl;
    }
}
```

```
    return 0;
```

```
}
```

/*Output

Please enter the size of the array:

7

Please enter the elements of the array:

10 80 30 90 40 50 70

Please enter the pivot value:

50

Array after partitioning around pivot 50:

10 30 40 50 80 70 90

Pivot placed at index: 3

***/**

Task 3:

3. Count the number of swaps required to sort an array using quick sort.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
long long swapCount = 0;
```

```
int Partition(vector<int> &arr, int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
    {
```

```
        if (arr[j] < pivot)
```

```
        {
```

```
            i++;
```

```
            if (i != j)
```

```
            {
```

```
                swap(arr[i], arr[j]);
```

```
                swapCount++;
```

```

        }
    }
}
swap(arr[i + 1], arr[high]);
swapCount++;
return i + 1;
}

```

```

void Quick_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = Partition(arr, low, high);
        Quick_Sort(arr, low, pivotIndex - 1);
        Quick_Sort(arr, pivotIndex + 1, high);
    }
}

```

```

int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;

```

```
cin >> N;
```

```
vector<int> arr(N);
```

```
cout << "Please enter the elements of the array: " << endl;
```

```
for (int i = 0; i < N; i++)
```

```
    cin >> arr[i];
```

```
Quick_Sort(arr, 0, N - 1);
```

```
cout << "Number of swaps: " << swapCount << endl;
```

```
cout << "Sorted array: " << endl;
```

```
for (int i = 0; i < N; i++)
```

```
    cout << arr[i] << " ";
```

```
cout << endl;
```

```
return 0;
```

```
}
```

/*Output

Please enter the size of the array:

5

Please enter the elements of the array:

8 4 7 3 5

Number of swaps: 5

Sorted array:

3 4 5 7 8

*/

Task 4:

4. Output the pivot chosen at each recursive call in quick sort (always pick the last element as pivot).

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int Partition(vector<int> &arr, int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    cout << "Pivot: " << pivot << endl;
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```

{
    if (arr[j] < pivot)
    {
        i++;
        swap(arr[i], arr[j]);
    }
}
swap(arr[i + 1], arr[high]);
return i + 1;
}

```

```

void Quick_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = Partition(arr, low, high);
        Quick_Sort(arr, low, pivotIndex - 1);
        Quick_Sort(arr, pivotIndex + 1, high);
    }
}

```

```

int main(void)

```

```

{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " << endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    Quick_Sort(arr, 0, N - 1);

    cout << "Sorted array: " << endl;
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

/*Output

Please enter the size of the array:

6

Please enter the elements of the array:

10 7 8 9 1 5

Pivot: 5

Pivot: 7

Pivot: 8

Pivot: 9

Sorted array:

1 5 7 8 9 10*/

Task 5:

5. For each recursive quick sort call, output the final index where pivot is placed.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int Partition(vector<int> &arr, int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
{
```

```

        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

```

```

void Quick_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = Partition(arr, low, high);
        cout << "Final index of pivot: " << pivotIndex << endl;
        Quick_Sort(arr, low, pivotIndex - 1);
        Quick_Sort(arr, pivotIndex + 1, high);
    }
}

```

```

int main(void)

```



```

{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " << endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    Quick_Sort(arr, 0, N - 1);

    cout << "Sorted array: " << endl;
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
/* output

```

Please enter the size of the array:

6

Please enter the elements of the array:

10 7 8 9 1 5

Final index of pivot: 1

Final index of pivot: 2

Final index of pivot: 3

Final index of pivot: 4

Sorted array:

1 5 7 8 9 10*/

Task 6:

6. For each pivot selected in quick sort (last element as pivot), count how many elements are less than the pivot at that step.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int Partition(vector<int> &arr, int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int elements_less = 0;
```

```
for (int k = low; k < high; k++)
```

```
{
```

```
    if (arr[k] < pivot)
```

```
        elements_less++;
```

```
}
```

```
cout << "Pivot " << pivot << " has " << elements_less
```

```
    << " element(s) less than it in this subarray." << endl;
```

```
int i = low - 1;
```

```
for (int j = low; j < high; j++)
```

```
{
```

```
    if (arr[j] < pivot)
```

```
    {
```

```
        i++;
```

```
        swap(arr[i], arr[j]);
```

```
    }
```

```
}
```

```
swap(arr[i + 1], arr[high]);
```

```
return i + 1;
```

```
}
```

```

void Quick_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = Partition(arr, low, high);
        Quick_Sort(arr, low, pivotIndex - 1);
        Quick_Sort(arr, pivotIndex + 1, high);
    }
}

```

```

int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " << endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    Quick_Sort(arr, 0, N - 1);
}

```

```
    cout << "Sorted array: " << endl;
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

/* Output

Please enter the size of the array:

5

Please enter the elements of the array:

8 4 7 3 5

Pivot 5 has 2 element(s) less than it in this subarray.

Pivot 3 has 0 element(s) less than it in this subarray.

Pivot 7 has 0 element(s) less than it in this subarray.

Sorted array:

3 4 5 7 8*/

Task 7:

7. Given two sorted arrays, find the median of the combined sorted array using merge sort.

Also print the merged array.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
vector<int> mergeArrays(const vector<int> &a, const  
vector<int> &b)
```

```
{
```

```
    vector<int> merged;
```

```
    int i = 0, j = 0;
```

```
    while (i < (int)a.size() && j < (int)b.size())
```

```
    {
```

```
        if (a[i] <= b[j])
```

```
            merged.push_back(a[i++]);
```

```
        else
```

```
            merged.push_back(b[j++]);
```

```
    }
```

```
    while (i < (int)a.size())
```

```
        merged.push_back(a[i++]);
while (j < (int)b.size())
    merged.push_back(b[j++]);

return merged;
}
```

```
double findMedian(const vector<int> &merged)
{
    int n = merged.size();
    if (n % 2 == 1)
        return merged[n / 2];
    else
        return (merged[n / 2 - 1] + merged[n / 2]) / 2.0;
}
```

```
int main(void)
{
    int N1, N2;

    cout << "Please enter the size of the first sorted array: "
<< endl;

    cin >> N1;
```

```
vector<int> a(N1);
```

```
cout << "Please enter the elements of the first sorted  
array: " << endl;
```

```
for (int i = 0; i < N1; i++)
```

```
    cin >> a[i];
```

```
    cout << "Please enter the size of the second sorted  
array: " << endl;
```

```
    cin >> N2;
```

```
vector<int> b(N2);
```

```
    cout << "Please enter the elements of the second sorted  
array: " << endl;
```

```
    for (int i = 0; i < N2; i++)
```

```
        cin >> b[i];
```

```
vector<int> merged = mergeArrays(a, b);
```

```
cout << "Merged array: " << endl;
```

```
for (int x : merged)
```

```
    cout << x << " ";
```

```
cout << endl;
```



```

    double median = findMedian(merged);
    cout << "Median of combined array: " << median << endl;

    return 0;
}
/*Output
Please enter the size of the first sorted array:
3
Please enter the elements of the first sorted array:
1 4 8
Please enter the size of the second sorted array:
4
Please enter the elements of the second sorted array:
2 5 6 9
Merged array:
1 2 4 5 6 8 9
Median of combined array: 5
*/

```

Task 8 :

8. Calculate the total number of comparisons made during merge sort.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
long long compareCount = 0;
```

```
void Merge(vector<int> &arr, int low, int mid, int high)
```

```
{
```

```
    vector<int> left(arr.begin() + low, arr.begin() + mid + 1);
```

```
    vector<int> right(arr.begin() + mid + 1, arr.begin() + high  
+ 1);
```

```
    int i = 0, j = 0, k = low;
```

```
    while (i < (int)left.size() && j < (int)right.size())
```

```
    {
```

```
        compareCount++;
```

```
        if (left[i] <= right[j])
```

```
            arr[k++] = left[i++];
```

```
        else
```

```
            arr[k++] = right[j++];
```

```
    }
```

```

while (i < (int)left.size())
    arr[k++] = left[i++];
while (j < (int)right.size())
    arr[k++] = right[j++];
}

```

```

void Merge_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;
        Merge_Sort(arr, low, mid);
        Merge_Sort(arr, mid + 1, high);
        Merge(arr, low, mid, high);
    }
}

```

```

int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;

```

```
cin >> N;
```

```
vector<int> arr(N);
```

```
cout << "Please enter the elements of the array: " <<  
endl;
```

```
for (int i = 0; i < N; i++)
```

```
    cin >> arr[i];
```

```
Merge_Sort(arr, 0, N - 1);
```

```
cout << "Number of comparisons: " << compareCount <<  
endl;
```

```
cout << "Sorted array: " << endl;
```

```
for (int i = 0; i < N; i++)
```

```
    cout << arr[i] << " ";
```

```
cout << endl;
```

```
return 0;
```

```
}
```

```
/*Output
```

```
Please enter the size of the array:
```

6

Please enter the elements of the array:

12 11 13 5 6 7

Number of comparisons: 9

Sorted array:

5 6 7 11 12 13

*/

Task 9:

9. Count the total number of merge operations performed when sorting an array using merge sort.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
long long mergeCount = 0;
```

```
void Merge(vector<int> &arr, int low, int mid, int high)
```

```

{
    mergeCount++;

    vector<int> left(arr.begin() + low, arr.begin() + mid + 1);
    vector<int> right(arr.begin() + mid + 1, arr.begin() + high
+ 1);

    int i = 0, j = 0, k = low;

    while (i < (int)left.size() && j < (int)right.size())
    {
        if (left[i] <= right[j])
            arr[k++] = left[i++];
        else
            arr[k++] = right[j++];
    }
    while (i < (int)left.size())
        arr[k++] = left[i++];
    while (j < (int)right.size())
        arr[k++] = right[j++];
}

```

```
void Merge_Sort(vector<int> &arr, int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;
        Merge_Sort(arr, low, mid);
        Merge_Sort(arr, mid + 1, high);
        Merge(arr, low, mid, high);
    }
}
```

```
int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " <<
endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];
}
```

```
Merge_Sort(arr, 0, N - 1);
```

```
cout << "Number of merge operations: " << mergeCount  
<< endl;
```

```
cout << "Sorted array: " << endl;
```

```
for (int i = 0; i < N; i++)
```

```
    cout << arr[i] << " ";
```

```
cout << endl;
```

```
return 0;
```

```
}
```

```
/*Output
```

```
Please enter the size of the array:
```

```
5
```

```
Please enter the elements of the array:
```

```
38 27 43 3 9
```

```
Number of merge operations: 4
```

```
Sorted array:
```

```
3 9 27 38 43
```

```
*/
```


Task 10:

10. Determine the total number of recursive calls merge sort makes for a given array.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
long long recursiveCallCount = 0;
```

```
void Merge(vector<int> &arr, int low, int mid, int high)
```

```
{
```

```
    vector<int> left(arr.begin() + low, arr.begin() + mid + 1);
```

```
    vector<int> right(arr.begin() + mid + 1, arr.begin() + high  
+ 1);
```

```
    int i = 0, j = 0, k = low;
```

```
    while (i < (int)left.size() && j < (int)right.size())
```

```
{
```

```
    if (left[i] <= right[j])
```

```
        arr[k++] = left[i++];
```

```

        else
            arr[k++] = right[j++];
    }
    while (i < (int)left.size())
        arr[k++] = left[i++];
    while (j < (int)right.size())
        arr[k++] = right[j++];
}

void Merge_Sort(vector<int> &arr, int low, int high)
{
    recursiveCallCount++;

    if (low < high)
    {
        int mid = low + (high - low) / 2;
        Merge_Sort(arr, low, mid);
        Merge_Sort(arr, mid + 1, high);
        Merge(arr, low, mid, high);
    }
}

```

```
int main(void)
{
    int N;
    cout << "Please enter the size of the array: " << endl;
    cin >> N;

    vector<int> arr(N);
    cout << "Please enter the elements of the array: " <<
endl;
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    Merge_Sort(arr, 0, N - 1);

    cout << "Number of recursive calls: " <<
recursiveCallCount << endl;

    cout << "Sorted array: " << endl;
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;
```

```
    return 0;  
}
```

/*Output

Please enter the size of the array:

4

Please enter the elements of the array:

42 17 85 23

Number of recursive calls: 7

Sorted array:

17 23 42 85

***/**